Ausgangssituation

Ein international tätiges Infrastruktur- und Energieunternehmen plant langfristige Investitionen in neue Standorte innerhalb Europas. Da die geplanten Anlagen eine Lebensdauer von mehreren Jahrzehnten haben, benötigt das Management eine fundierte datenbasierte Einschätzung darüber, wie sich klimatische Bedingungen in verschiedenen europäischen Regionen entwickelt haben und welche Trends für die Zukunft erkennbar sind.

Zielsetzung

Ziel dieses Projekts ist es, anhand realer internationaler Klimadaten eine strukturierte Datenanalyse durchzuführen und daraus wirtschaftlich relevante Erkenntnisse abzuleiten. Der Schwerpunkt liegt dabei nicht auf einer wissenschaftlich-methodischen Vorgehensweise, sondern auf dem praktischen Umgang mit Daten – insbesondere deren Beschaffung, Aufbereitung, Analyse und Visualisierung.

Datenbeschaffung

Für die Datenbeschaffung wird R verwendet, welches mit WDI über eine perfekte Schnittstelle zur World Bank verfügt. So können die Datenbestände einfach per ID über den Befehl `WDIsearch("ID", field = "indicator")` gesucht und später abgerufen werden. Hier entsteht jedoch bereits das erste Problem: Datensilo. Die Indikatoren sind in unterschiedlichen Hauptdatenbanken abgespeichert. Während Der Indikator „Average precipitation in depth (mm per year)“ in der Hauptdatenbank „World Development Indicators“ abgespeichert und per WDI erreicht werden kann, befindet sich der Indikator „Average Mean Temperature (°C)“ im CCKP-Silo (Climate Change Knowledge Portal) und kann daher nicht per WDI abgerufen werden. Es besteht nun die Möglichkeit (a) einen eigenen API-Abruf zu schreiben oder (b) die Daten per CSV herunterzuladen. Da hier ein Lern-Fokus gelegt wird entscheide ich mich für Option a. Daraus ergeben sich folgende Befehle für den Abruf:

```
niederschlag_daten <- WDI(country = c("DE", "ES", "SE", "FR", "RS"),
                          indicator = "AG.LND.PRCP.MM",
                          start = 1990,
                          end = 2021)
```

Durch den Befehl `head(niederschlag_daten)` lasse ich mir zudem die ersten 6 Zeilen anzeigen, um zu überprüfen, ob der Abruf korrekt funktioniert hat. Hierbei fällt mir auf, dass die Wertangabe für jedes Land über den Betrachtungszeitraum konstant bzw. statisch ist:

	country	iso2c	iso3c	year	AG.LND.PRCP.MM Average precipitation in depth (mm per year)
1	Germany	DE	DEU	2021	700
2	Germany	DE	DEU	2020	700
3	Germany	DE	DEU	2019	700
4	Germany	DE	DEU	2018	700
5	Germany	DE	DEU	2017	700
6	Germany	DE	DEU	2016	700
7	Germany	DE	DEU	2015	700
8	Germany	DE	DEU	2014	700
9	Germany	DE	DEU	2013	700
10	Germany	DE	DEU	2012	700
11	Germany	DE	DEU	2011	700
12	Germany	DE	DEU	2010	700
13	Germany	DE	DEU	2009	700
14	Germany	DE	DEU	2008	700
15	Germany	DE	DEU	2007	700
16	Germany	DE	DEU	2006	700
17	Germany	DE	DEU	2005	700
18	Germany	DE	DEU	2004	700
19	Germany	DE	DEU	2003	700

Meine Schlussfolgerung: Der technische Abruf hat einwandfrei funktioniert, jedoch passt die fachliche Natur nicht, da bei der Datenerhebung der „long-term average“ (also der langfristige, historische Durchschnitt) verwendet wurde. Eine Lösung muss gefunden werden…

Als Alternative wird der Datenabruf beider Indiaktoren nun durch einen eigenen API-Abruf aus dem CCKP-Silo umgesetzt. In Anbetracht der Zielsetzung ist dies die beste Lösung.

```r
library(jsonlite)
library(dplyr)

laender_codes <- c("DEU", "ESP", "SWE", "FRA", "SRB")
niederschlag_alle <- data.frame()

for (land in laender_codes) {
  api_url <- paste0("https://climateknowledgeportal.worldbank.org/api/v1/cru-x0.5/pr/timeseries/historical/annual/mean/1901-2022/", land, "?format=json")

  rohdaten <- fromJSON(api_url)

  df <- as.data.frame(rohdaten$data)
  niederschlag_alle <- bind_rows(niederschlag_alle, df)
}

niederschlag_final <- niederschlag_alle %>%
  filter(year >= 1990)

head(niederschlag_final)
```

Neues Problem identifiziert: Es scheint, als wäre die textbasierte Web-API der World Bank abgeschaltet. Die Daten können nur noch aus komplexen Data-cubes über den Cloud-Speicher

Amazon Web Services abgefragt werden. Ich muss zunächst prüfen, ob dies den Umfang dieses Projektes überschreitet.

Erkenntnis: Ja, das sprengt definitiv den Rahmen. Der Umgang mit mehrdimensionalen Geodaten-Rastern gehört in die Geoinformatik und lenkt vom eigentlichen Projektziel ab. Daher muss erneut eine Lösung gesucht werden...

Ich habe entdeckt, dass auf der Website des Climate Change Knowledge Portal eine vollstände API-URL erstellt werden kann. Diese ist aufgebaut wie ein Baukasten bestehend aus den verschiedenen Parametern, welche ich für jeden Land auswählen kann.

Beispiel-Parameter

Area of Focus CHANGE

France (FRA)

Collection CHANGE

CRU 0.5-degree (cru-x0.5)

Type CHANGE

Timeseries (timeseries)

Variable CHANGE

Average Mean Surface Air Temperature (tas)
Precipitation (pr)

Product CHANGE

Time Series (timeseries)

Aggregation CHANGE

Annual (annual)

Period CHANGE

1901-2024 (1901-2024)

Percentile CHANGE

Mean - Individual Model Access (mean)

Scenario CHANGE

Historical (historical)

Model CHANGE

Cru (cru)

Model Calculation CHANGE

Ts4.09 (ts4.09)

Statistic CHANGE

Mean (mean)

Beispiel API-URL: https://cckpapi.worldbank.org/api/v1/cru-x0.5_timeseries_tas,pr_timeseries_annual_1901-2024_mean_historical_cru_ts4.09_mean/FRA?_format=json

Ich habe bereits einen Testlauf in Rstudio durchgeführt und die URL funktioniert tatsächlich. Alle Daten werden korrekt eingelesen. Dies ist der Schlüssel für den API-Abruf. Die vorgegebene API-URL kann nun als eine Art Generalschlüssel verwendet werden, indem der Teil „FRA“ für die jeweiligen Länder-Abkürzungen „SWE“, „SP“ usw. ersetzt wird. Dies ermöglicht zudem eine for-Schleife, wodurch das Skript deutlich an Effizienz gewinnt.

```
library(jsonlite)
library(dplyr)

laender_codes <- c("DEU", "ESP", "SWE", "FRA", "SRB")
klima_master <- data.frame()

for (land_code in laender_codes) {


  dynamische_url <- paste0("https://cckpapi.worldbank.org/api/v1/cru-x0.5_timeseries_tas,pr_timeseries_annual_1901-2024_mean_historical_cru_ts4.09_mean/", land_code, "?_format=json")

  roh <- fromJSON(dynamische_url)
  df <- if ("data" %in% names(roh)) as.data.frame(roh$data) else as.data.frame(roh)

  klima_master <- bind_rows(klima_master, df)
}

klima_final <- klima_master %>%
  filter(year >= 1990 & year <= 2021)

head(klima_final)
```

Bei Ausführung des Skripts entsteht eine Fehlermeldung. Der Jahres-Filter verursacht einen error. Zudem wurden nicht alle Datensätze vollständig eingelesen. Der Grund hierfür ist, dass die API die Datenbestände im Wide-Format liefert (ein Land pro Zeile und hunderte Jahres-Spalten). Daher ist zunächst eine Datentransformation in ein sauberes Long-Format nötig. Hierzu verwende ich die Funktion pivot_longer aus dem tidyr-Paket, um aus den Jahresspalten die 3 Spalten „Country“, „Variable“ und „Value“ zu machen. Nach einigen Fehlversuchen bin ich zu folgender Lösung gekommen:

```
library(jsonlite)
library(dplyr)
library(tidyr)
library(stringr)

laender_codes <- c("DEU", "ESP", "SWE", "FRA", "SRB")
klima_master <- data.frame()

for (land_code in laender_codes) {

  dynamische_url <- paste0("https://cckpapi.worldbank.org/api/v1/cru-
x0.5_timeseries_tas,pr_timeseries_annual_1901-
2024_mean_historical_cru_ts4.09_mean/", land_code, "?_format=json")
  roh <- fromJSON(dynamische_url)
  df <- if ("data" %in% names(roh)) as.data.frame(roh$data) else
as.data.frame(roh)

  df_long <- df %>%
    pivot_longer(
      cols = everything(),
      names_to = "roh_name",
      values_to = "wert"
```

```
    ) %>%
    separate(roh_name, into = c("variable", "country", "year", "monat_drop"),
sep = "\\.") %>%
    select(-monat_drop) %>%
    mutate(year = as.numeric(year)) %>%
    filter(year >= 1990 & year <= 2022)

  klima_master <- bind_rows(klima_master, df_long)
}

klima_final <- klima_master %>%
  pivot_wider(names_from = variable, values_from = wert)

head(klima_final, 10)

write.csv(klima_final, "Klimadaten_Clean_1990_2022.csv", row.names = FALSE)
```

Hierbei bin ich auf verschiedene Probleme sowie Lösungen gestoßen, welche ich folgend erklären möchte.

1. Im alten Code habe ich versucht direkt nach dem API-Abruf eine Spalte namens „year“ zu filtern, welche es so gar nicht gab. Denn die World Bank liefert die Daten im bereits genannten Wide-Format, wobei jedes Jahr eine eigene Spalte war (z.B. pr.DEU.1990.07 oder tas.DEU.1991.07)
2. `pivot_longer(cols = everything(), names_to = "roh_name", values_to = "wert")`

   Da die gegebene Struktur des Datenbestandes grundlegend geändert werden muss, wird die Tabelle zunächst „eingeschmolzen“ und von einem Wide-Format in ein Long-Format umstrukturiert. R nimmt dabei die 248 Spalten und schreibt die alten Spaltennamen in eine neue Spalte namens `roh_name`. Die darin enthaltenen Messwerte wandern in die Spalte `wert`. Aus 1 Zeile x 248 Spalten werden dadurch 248 Zeilen x 2 Spalten. Hier eine kurze Darstellung:

   Vorher:

	pr.DEU.1989.07	pr.DEU.1990.07	pr.DEU.1991.07	...	tas.DEU.1989.07	tas.DEU.1990.07
1	701.2	712.5	690.8	...	9.1	10.2

   Nachher:

	roh_name	wert
1	pr.DEU.1989.07	701.2
2	pr.DEU.1990.07	712.5
3	pr.DEU.1991.07	690.8
4	...	...
5	tas.DEU.1989.07	9.1
6	tas.DEU.1990.07	10.2

3. `separate(roh_name, into = c("variable", "country", "year", "monat_drop"), sep = "\\.")`

   Nun muss der Text in roh_name in seine Einzelteile zersetzt werden. Dafür sucht die Funktion seperate nach einem Trennzeichen (hier: „.“). Sie zerschneidet den Text (z.B. „pr.DEU.1989.07“) in vier Teile und weist diese den neuen Spaltennamen „variable“, „country“, „year“ und „monat_drop“ zu. Aus der Spalte `roh_name` wurden dadurch 4

neue Spalten. Das darauffolgende `select(-monat_drop)` ist lediglich eine Bereinigungsmaßnahme, da der Monat (07 steht für Juli) nicht für die Jahresanalyse benötigt wird. Folgend das Zwischenergebnis:

	variable	country	year	wert
1	pr	DEU	1989	701.2
2	pr	DEU	1990	712.5
3	pr	DEU	1991	690.8
4	...	...	...	...
5	tas	DEU	1989	9.1
6	tas	DEU	1990	10.2

4. `mutate(year = as.numeric(year))`

   Da die Werte der Spalte year aus dem Herausschneiden aus einem Text entstanden sind (siehe Punkt 3), denkt R aktuell, dass es sich bei den Jahresangaben um ein Wort (String) handelt. Damit spätere Rechnungen oder Vergleiche möglich sind bringe durch `as.numeric` R dazu, diese Spalte ab sofort als echte Zahl zu behandeln.

5. `filter(year >= 1990 & year <= 2022)`

   Dieser Filter ist selbsterklärend. Er beschränkt die Datenreihe auf den zuvor festgelegten Betrachtungszeitraum (von 1990 bis 2022).

6. `pivot_wider(names_from = variable, values_from = wert)`

   Bei dem bisherigen Zwischenergebnis besteht ein Strukturproblem für den späteren Import in Power BI, denn Temperatur (tas) und Niederschlag (pr) stehen abwechselnd in derselben Spalte (variable). Power BI benötigt diese Metriken zwingend in eigenen, separaten Spalten. Durch diesen Befehl nimmt R die Einträge aus der Spalte variable (also pr und tas) und macht aus diesen neue Spaltenköpfe. Danach werden diese Spalten mit den Zahlen aus der Spalte wert gefüllt. Folgend das Endergebnis:

	country	year	pr	tas
1	DEU	1990	712.5	10.2
2	DEU	1991	690.8	9.8

Anmerkung: Bei den Darstellungen handelt es sich nur um einen Ausschnitt des tatsächlichen Datenbestandes

Qualitätsüberprüfung

Nun führe ich einige Qualitätschecks durch, um zu überprüfen, ob sich durch die Datentransformationen Fehler eingeschlichen haben, da ich diese später nicht in meine Power-BI-Visualisierungen übernehmen möchte. Folgende Aspekte möchte ich dabei untersuchen:

1. Datentypen-Prüfung

   Durch das Umbauen der Tabellen könnte R durch ein fehlerhaftes Komma oder Leerzeichen versehentlich eine Zahl als Text (String) verstehen. Ich möchte daher prüfen, ob die Spalten year, tas und pr als numeric bzw. integer geführt werden. Diese Prüfung führe ich in 2 Phasen durch: `glimpse()` für die Überprüfung bzw. Diagnose

und `mutate()` für die Korrektur (falls nötig). Zunächst führe ich also `glimpse(klima_final)` aus:

```
$ country <chr> "DEU", "DEU", "DE
$ year    <dbl> 1990, 1991, 1992
$ pr      <dbl> 733.92, 604.67, 7
$ tas     <dbl> 9.81, 8.72, 9.64
```

Die Ausgabe zeigt, dass Spalte country als chr (character/string) und die Spalten year, pr und tas als dbl (double/numeric) erkannt werden. Das ist grundsätzlich vollkommen in Ordnung, jedoch könnte Spalte year ebenfalls als int (integer/Ganzzahl) erkannt werden, da das Jahr natürlich immer eine ganze Zahl ist. Aus Demonstrationsgründen möchte ich daher den dortigen Datentyp ändern. Ich führe also den Befehl `klima_final <- klima_final %>% mutate(year = as.integer(year))` aus. Folgend das Ergebnis:

```
$ country <chr> "DEU", "DEU", "D
$ year    <int> 1990, 1991, 1992
$ pr      <dbl> 733.92, 604.67,
$ tas     <dbl> 9.81, 8.72, 9.64
```

2. Fehlende Werte

Fehlende Werte sind in der Datenanalyse ein riesiges Problem, da R eine leere Zelle als striktes Konzept (NA; Not Available) interpretiert. Der Durchschnitt von 10, 15 und NA wäre daher auch wieder NA. In der Praxis müsste man solche leeren Zellen durch Verfahren wie die lineare Regression auffüllen. Ich möchte mich nun lediglich mit der Identifikation solcher Zellen beschäftigen. Dies geschieht mit der Funktion `is.na()`, welche jede Zelle durchläuft und TRUE oder FALSE ausgibt. Da R intern True als 1 und FALSE als 0 wertet, können die Werte einfach mit dem Befehl `colSums(is.na())` aufsummiert werden. Das Ergebnis gibt den Aufschluss über die Anzahl der leeren Zellen (NA) für jede Spalte.

```
> is.na(klima_final)
       country  year    pr   tas
  [1,]   FALSE FALSE FALSE FALSE
  [2,]   FALSE FALSE FALSE FALSE
  [3,]   FALSE FALSE FALSE FALSE
  [4,]   FALSE FALSE FALSE FALSE
  [5,]   FALSE FALSE FALSE FALSE
  [6,]   FALSE FALSE FALSE FALSE
  [7,]   FALSE FALSE FALSE FALSE
  [8,]   FALSE FALSE FALSE FALSE
  [9,]   FALSE FALSE FALSE FALSE
 [10,]   FALSE FALSE FALSE FALSE
 [11,]   FALSE FALSE FALSE FALSE
```

```
> colSums(is.na(klima_final))
country    year      pr     tas
      0       0       0       0
```

Das Ergebnis ist wie erwartet. Mein Datenbestand enthält keine leeren Zellen.

3. Plausibilitäts-Checks

Hier möchte ich meinen Datenbestand auf inhaltliche Fehler oder Ungereimtheiten untersuchen. Dafür habe ich mir 2 Funktionen überlegt.

Zunächst möchte ich den Befehl `summary(klima_final)` durchführen, welcher für jede numerische Spalte wie tas oder pr das Minimum, Maximum sowie den Mittelwert angibt. So können unmögliche Werte wie negativer Niederschlag (z.B. -50 in Spalte pr) oder unrealistisch hohe Temperaturen (z.B. 90 in Spalte tas) identifiziert werden.

```
       pr              tas
 Min.   : 464.1   Min.   : 1.33
 1st Qu.: 630.4   1st Qu.: 9.20
 Median : 712.7   Median :11.10
 Mean   : 715.6   Mean   :10.03
 3rd Qu.: 805.7   3rd Qu.:12.39
 Max.   :1019.8   Max.   :15.16
```

Die ausgegebenen Werte sind realistisch, wodurch keine inhaltlichen Fehler zu erwarten sind. In komplexeren Kontexten wäre dies keine ausreichende Qualitätsmaßnahme anzusehen. Hier soll es jedoch ausreichen.

Zusätzlich möchte ich mit dem Befehl `unique(klima_final$country)` überprüfen, ob mir eventuell andere Ländercodes in die Spalte country gerutscht sind.

```
> unique(klima_final$country)
[1] "DEU" "ESP" "SWE" "FRA" "SRB"
```

Das Ergebnis zeigt, dass sich keine ungewollten Länder(-codes) in meinem Datenbestand befinden.

Damit soll der Aspekt der Qualitätssicherung hiermit abgeschlossen sein. Mein Datenbestand befindet sich nun in einem transformierten, bereinigten sowie qualitätsgesicherten Zustand und ist perfekt für den Import in Power BI vorbereitet.

Überführung in Power-BI

Als nächstes folgt der Import der Daten in Power BI, um dort die Visualisierung des Datenbestandes durchzuführen. Hierfür muss ich jedoch einige Dinge beachten, denn CSV-Dateien aus R verwenden den internationalen Standard (Punkt als Komma, z.B. 10.2). Power BI erwartet jedoch ein Komma (z.B. 10,2). Um kaputte Datentypen zu vermeiden, muss ich Power BI darauf hinweisen, dass die Zahlen „auf Englisch“ eingelesen werden sollen.

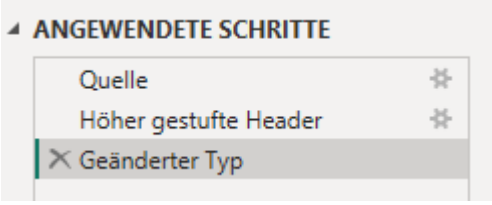
Ich importiere daher meine finale Klimadaten-CSV in Power BI und navigiere dort auf „Daten transformieren“, wodurch ich zum Power Query-Editor gelange. Hier ändere ich den Datentyp der Spalten pr und tas auf „Dezimalzahl“ mit dem Gebietsschema „Englisch (USA)“. Nun sollten alle Werte korrekt erkannt werden.

Problem: Die Änderung des Datentyps hatte nicht die gewollte Wirkung. Die Werte von pr und tas werden immer noch fehlerhaft erkannt.

	country	year	pr	tas
1	DEU	1990	73392	981
2	DEU	1991	60467	872
3	DEU	1992	7349	964
4	DEU	1993	79351	875
5	DEU	1994	78935	993
6	DEU	1995	78201	914
7	DEU	1996	62283	751
8	DEU	1997	6658	913
9	DEU	1998	80947	929
10	DEU	1999	76928	975
11	DEU	2000	75817	1014
12	DEU	2001	81309	92
13	DEU	2002	91079	975
14	DEU	2003	56828	951
15	DEU	2004	73182	917

Jedoch habe ich die Fehlerquelle vermutlich schon identifiziert. Beim Import bzw. Transformation der Daten wendet Power BI automatisch die deutsche Logik an und „rät“ die Werte. Daher wird „9.81“ zu „981“. Meine Änderung des Datentyps und die Anpassung des Gebietsschemas wird dann auf die bereits fehlerhafte Zahl angewendet. Ich muss also den Zwischenschritt von Power BI unterbinden.

Ich öffne also erneut Power BI und navigiere in den Power Query-Editor. Hier wird mir der Fehler schnell deutlich:



Ich mache den letzten Schritt „Geänderter Typ“ (welcher automatisch durch Power BI durchgeführt wurde) rückgängig und führe erneut die Datentyp-Anpassung durch. Das Ergebnis ist meine gewünschte Tabelle mit den korrekten Werten. Alles ist bereit für den endgültigen Import in Power BI.

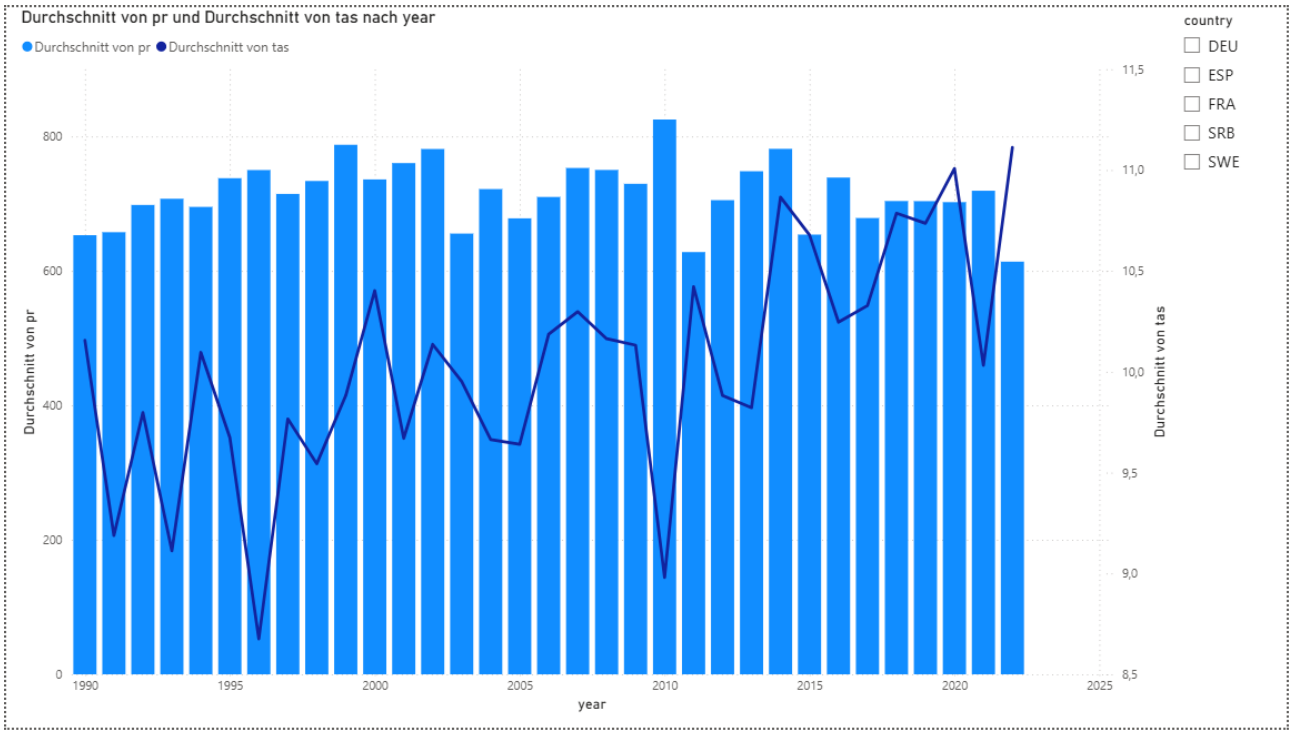
	country	year	pr	tas
1	DEU	1990	733,92	9,81
2	DEU	1991	604,67	8,72
3	DEU	1992	734,9	9,64
4	DEU	1993	793,51	8,75
5	DEU	1994	789,35	9,93
6	DEU	1995	782,01	9,14
7	DEU	1996	622,83	7,51
8	DEU	1997	665,8	9,13
9	DEU	1998	809,47	9,29
10	DEU	1999	769,28	9,75
11	DEU	2000	758,17	10,14
12	DEU	2001	813,09	9,2
13	DEU	2002	910,79	9,75
14	DEU	2003	568,28	9,51
15	DEU	2004	731,82	9,17

Edit: Ich habe noch den Datentyp der Spalte year von Text auf Ganzzahl geändert.

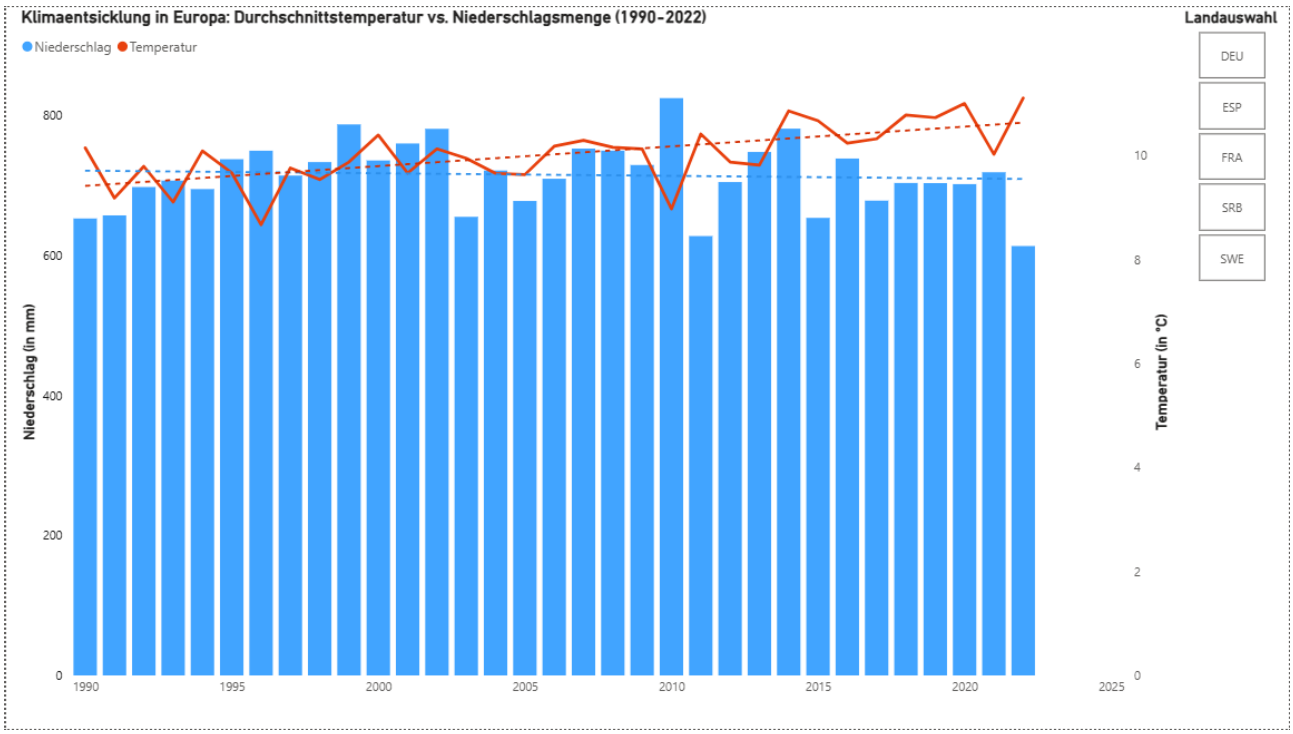
Für die Visualisierung wähle ich ein Linien- und gruppiertes Säulendiagramm, da ich den Niederschlag und die Temperatur gemeinsam darstellen möchte. Hierbei muss ich 2 zentrale Aspekte beachten:

1. Ich habe zwei verschiedene Maßeinheiten (Millimeter für Niederschlag, Grad Celsius für Temperatur). Mein gewähltes Diagramm unterstützt zwei separate y-Achsen.
2. Aggregations-Korrektur: Power BI neigt dazu Werte zu summieren. Beispiel: Power BI summiert die Werte 10 Grad (DEU), 15 Grad (ESP) und 5 Grad (SWE) und zeichnet für das entsprechende Jahr einen Datenpunkt bei 30 Grad. Inhaltlich/logisch ist das jedoch falsch. Ich zwinge Power BI daher dazu den Mittelwert zu verwenden.

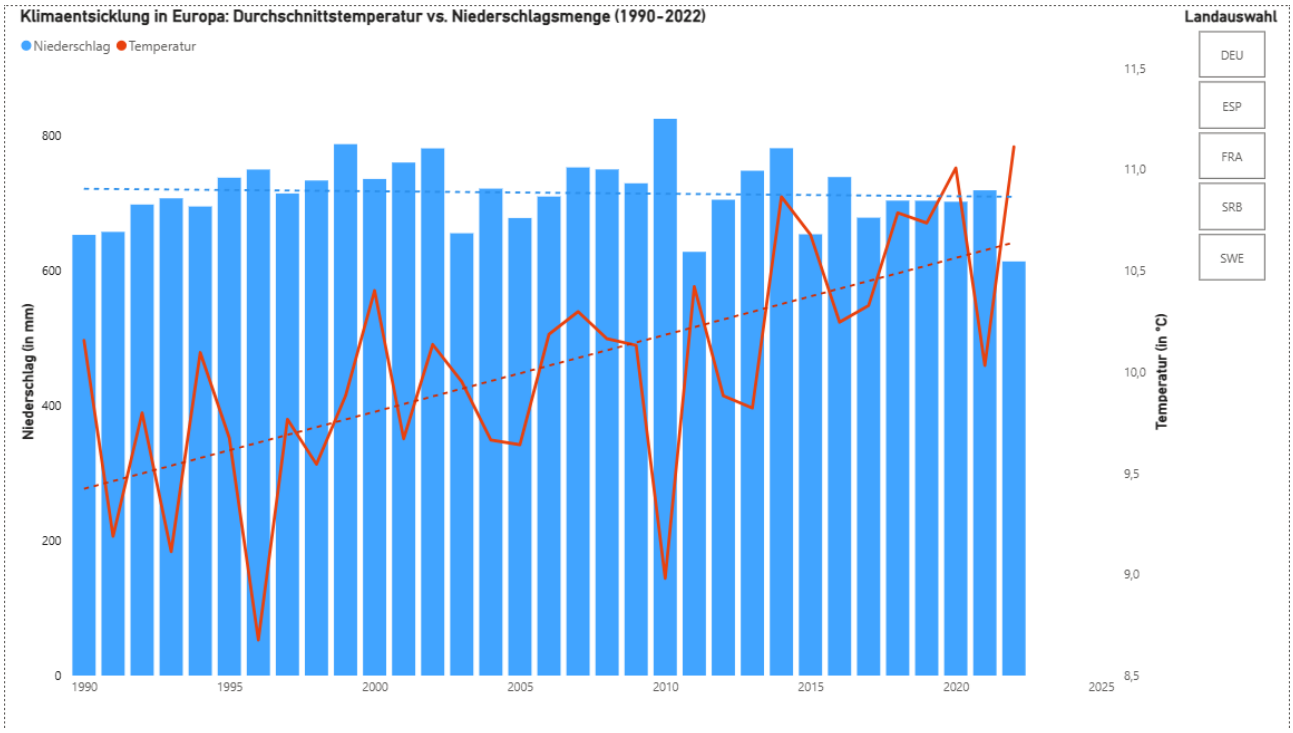
Das Grundgerüst meiner Visualisierung steht nun. Ich verwende die Spalte year auf der X-Achse, Durchschnitt von pr auf y-Achse (Spalte) und den Durchschnitt von tas auf der Y-Achse (Linie). Zudem füge ich einen Datenschnitt hinzu, wodurch die Visualisierung für jedes Land einzeln angezeigt werden kann.



Dies bildet die grobe Struktur, welche ich weiter anpassen möchte. An der Stelle gehe ich nicht detailliert auf die weitere Umsetzung des Visuals ein, jedoch habe ich Bezeichnungen verändert, einen Datenschnitt nach Land hinzugefügt und Trendlinien eingezeichnet. Das Ziel: Der Betrachter soll innerhalb von wenigen Sekunden alle Trends und Kernaspekte des Visuals erkennen können. Folgend meine finale Version des Visuals.



Ein zentrales Learning war dabei die Wirkung von Skalierungen auf die Wahrnehmung. Eine abgeschnittene sekundäre Y-Achse (Start bei z.B. 8,5 °C) ist mathematisch nicht falsch, kann Trends jedoch optisch überproportional steil erscheinen lassen (siehe: Trendlinie der Temperatur). Solche Formatierungsentscheidungen sollten bewusst getroffen werden.



Was habe ich gelernt?

- Grundkonzepte
  - %>% Pipe-Operator
  - Unterschied zwischen pivot_longer und pivot_wider
- Datentransformation
  - Rohdaten einer API strukturell umbauen
  - as.numeric() und korrekte Datentypen
- Qualitätssicherung
  - glimpse()
  - colSums(is.na())
  - summary() und unique()
  - Was NAs sind und wie R damit umgeht
- Power BI
  - Troubleshooting bezüglich Datentyp und Gebietsschema beim Datenimport
  - Umgang und Darstellung von Metriken in unterschiedlichen Einheiten (mm vs. °C)
  - Anpassung der visuellen Benutzerfreundlichkeit
  - Wirkung der Achsenskalierung auf Wahrnehmung